Course Title:

BIT302 Software Engineering

Course Contents

Unit 6: Design and Implementation (5 Hrs.)

Object-Oriented Design Using the UML,
Design Patterns,
Implementation Issues,
Open-Source Development

Course Contents

Unit 6: Design and Implementation

7.1 Object-Oriented Design Using the UML
- 7.1.1 System context and interactions
- 7.1.2 Architectural design
- 7.1.3 Object class identification
- 7.1.4 Design models
- 7.1.5 Interface specification

7.2 Design Patterns

7.3 Implementation Issues,
- 7.3.2 Configuration management
- 7.3.3 Host-target development

7.4 Open-Source Development
- 7.4.1 Open-source licensing

Course Contents

Design and Implementation- Objectives

You will

- understand the most important activities in a general, object-oriented design process;
- understand some of the different models that may be used to document an object-oriented design;
- know about the idea of design patterns and how these are a way of reusing design knowledge and experience;
- have been introduced to key issues that have to be considered when implementing software, including software reuse and open-source development.

Course Contents

Design And Implementation:

- Software design and implementation is the stage in the software engineering process at which an executable software system is developed.
- Software design and implementation activities are invariably inter-leaved. Software design is a creative activity in which you identify software components and their relationships, based on a customer's requirements.
- Implementation is the process of realizing the design as a program.
- Structured object-oriented design processes involve developing a number of different system models.
- They require a lot of effort for development and maintenance of these models and, for small systems, this may not be cost-effective.
- However, for large systems developed by different groups design models are an important communication mechanism.

Course Contents

Design And Implementation:

- Design is about how to solve a problem, so there is always a design process. However, it isn't always necessary or appropriate to describe the design in detail using the UML or other design description language.
- Design and implementation are closely linked, and you should normally take implementation issues into account when developing a design. For example, using the UML to document a design may be the right thing to do if you are programming in an object-oriented language such as Java or C#. It is less useful, I think, if you are developing using a dynamically typed language like Python. There is no point in using the UML if you are implementing your system by configuring an off-the-shelf package. Agile methods usually work from informal sketches of the design and leave design decisions to programmers.

Course Contents

Design And Implementation:

This chapter has two aims:

1. To show how system modeling and architectural design (covered earlier) are put into practice in developing an object-oriented software design.
2. To introduce important implementation issues that include software reuse, configuration management and open-source development.

As there are a vast number of different development platforms, presented here all examples using the UML rather than a programming language such as Java or Python.

Course Contents

An Object-oriented Design Process:

There are a variety of different object-oriented design processes that depend on the organization using the process. Common activities in these processes include:

1. Understand and define the context and the external interactions with the system.
2. Design the system architecture.
3. Identify the principal objects in the system.
4. Develop design models.
5. Specify interfaces.

Course Contents

System context and interactions:

- The first stage in any software design process is to develop an understanding of the relationships between the software that is being designed and its external environment. This is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment.
- Understanding of the context also lets you establish the boundaries of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed and what features are in other associated systems.

Course Contents

System context and interactions:

System context models and interaction models present complementary views of the relationships between a system and its environment:

1. A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
2. An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.

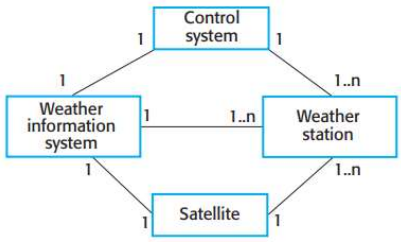


Figure 7.1 System context for the weather station

Course Contents

System context and interactions:

The context model of a system may be represented using associations. Associations simply show that there are some relationships between the entities involved in the association. You can document the environment of the system using a simple block diagram, showing the entities in the system and their associations.

Figure 7.1 shows that the systems in the environment of each weather station are a weather information system, an onboard satellite system, and a control system. The cardinality information on the link shows that there is a single control system but several weather stations, one satellite, and one general weather information system.

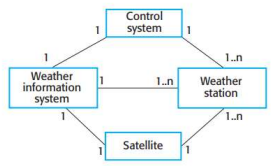


Figure 7.1 System context for the weather station

Course Contents

System context and interactions:

System context and interactions model does not include too much detail. One way to do this is to use a use case model. Each use case represents an interaction with the system. Each possible interaction is named in an ellipse, and the external entity involved in the interaction is represented by a stick figure.

The use case model for the weather station is shown in Figure 7.2.

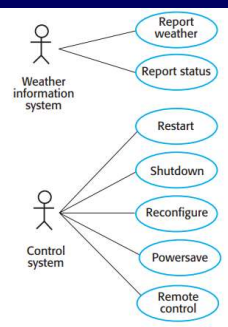


Figure 7.2 Weather station use cases

Course Contents

System context and interactions:

- This shows that the weather station interacts with the weather information system to report weather data and the status of the weather station hardware. Other interactions are with a control system that can issue specific weather station control commands. The stick figure is used in the UML to represent other systems as well as human users.
- Each of these use cases should be described in structured natural language. This helps designers identify objects in the system and gives them an understanding of what the system is intended to do.

Figure 7.3 Use case description—Report weather

System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Data	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum and average wind speeds; the total rainfall; and the wind direction as sampled at 5-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour, but this frequency may differ from one station to another and may be modified in future.

Course Contents

System context and interactions:

System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Description	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; the total rainfall; and the wind direction as sampled at five-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

Course Contents

Architectural design:

- Once the interactions between the software system and the system's environment have been defined, you use this information as a basis for designing the system architecture.
- You identify the major components that make up the system and their interactions. You may then design the system organization using an architectural pattern such as a layered or client–server model. The high-level architectural design for the weather station software is shown in Figure 7.4.

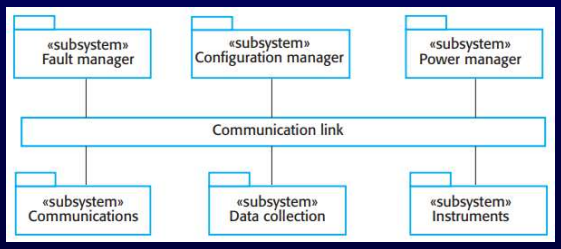


Figure 7.4 High-level architecture of weather station

Course Contents

Object class identification:

- By this stage in the design process, you should have some ideas about the essential objects in the system that you are designing. As your understanding of the design develops, you refine these ideas about the system objects.
- The use case description helps to identify objects and operations in the system. From the description of the Report weather use case, it is obvious that you will need to implement objects representing the instruments that collect weather data and an object representing the summary of the weather data.
- With these objects in mind, you can start to identify the general object classes in the system.

Course Contents

Object class identification:

- With these objects in mind, you can start to identify the general object classes in the system.

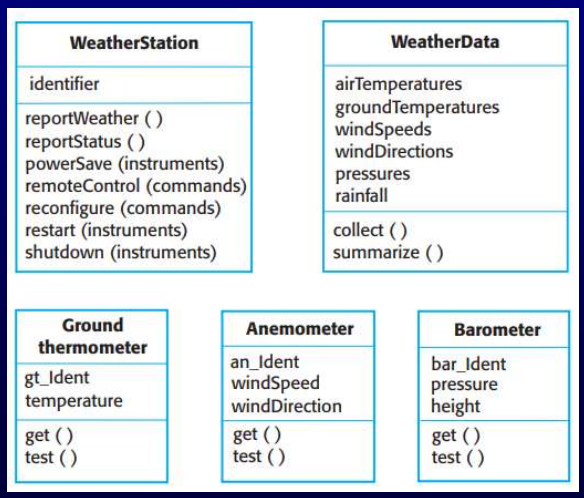


Figure 7.6 Weather station objects

Course Contents

Design models:

- Design or system models, as discussed before, show the objects or object classes in a system. They also show the associations and relationships between these entities. These models are the bridge between the system requirements and the implementation of a system. They have to be abstract so that unnecessary detail doesn't hide the relationships between them and the system requirements. However, they also have to include enough detail for programmers to make implementation decisions.

Course Contents

Design models:

UML model types are particularly useful for adding detail to use case and architectural models:

1. Subsystem models, which show logical groupings of objects into coherent subsystems. These are represented using a form of class diagram with each subsystem shown as a package with enclosed objects. Subsystem models are structural models.
2. Sequence models, which show the sequence of object interactions. These are represented using a UML sequence or a collaboration diagram. Sequence models are dynamic models.
3. State machine models, which show how objects change their state in response to events. These are represented in the UML using state diagrams. State machine models are dynamic models.

Course Contents

Design models:

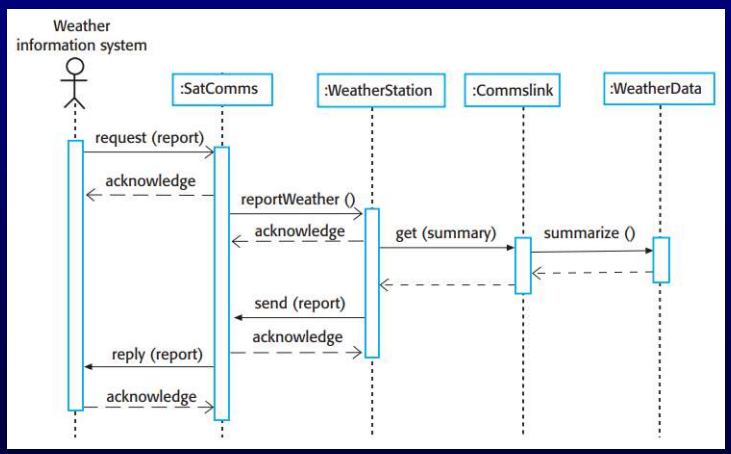


Figure 7.7 Sequence diagram describing data collection

Course Contents

Design models:

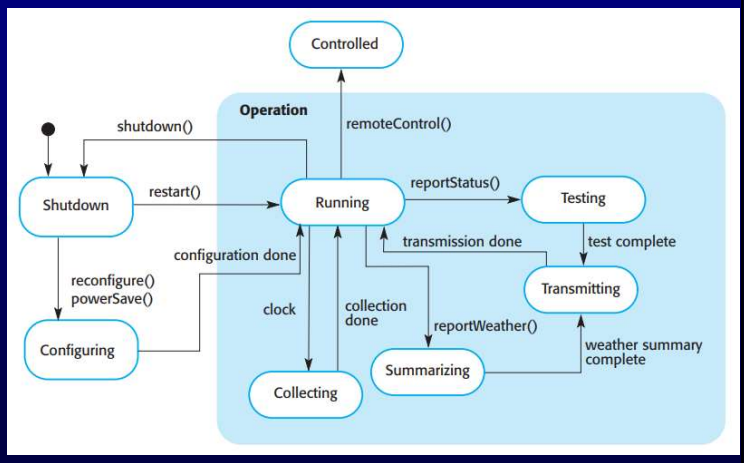


Figure 7.8 Weather station state diagram

Course Contents

Interface specification:

Interface design is concerned with specifying the detail of the interface to an object or to a group of objects. This means defining the signatures and semantics of the services that are provided by the object or by a group of objects. Interfaces can be specified in the UML using the same notation as a class diagram. However, there is no attribute section, and the UML stereotype «interface» should be included in the name part.

Figure 7.9 Weather station interfaces

Course Contents

Design Patterns:

Patterns are a way of reusing the knowledge and experience of other designers. Design patterns are usually associated with object-oriented design. The Gang of Four defined the four essential elements of design patterns in their book on patterns:

1. A name that is a meaningful reference to the pattern.
2. A description of the problem area that explains when the pattern may be applied.
3. A solution description of the parts of the design solution, their relationships and their responsibilities. This is not a concrete design description. It is a template for a design solution that can be instantiated in different ways. This is often expressed graphically and shows the relationships between the objects and object classes in the solution.
4. A statement of the consequences—the results and trade-offs—of applying the pattern. This can help designers understand whether or not a pattern can be used in a particular situation.

Course Contents

Implementation Issues:

- Software engineering includes all of the activities involved in software development from the initial requirements of the system through to maintenance and management of the deployed system.
- A critical stage of this process is, of course, system implementation, where you create an executable version of the software.
- Implementation may involve developing programs in high- or low-level programming languages or tailoring and adapting generic, off-the-shelf systems to meet the specific requirements of an organization.

Some aspects of implementation that are particularly important to software engineering and that are often not covered in programming texts are:

- Reuse
- Configuration management
- Host-target development

Course Contents

Implementation Issues:

Reuse- Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.

Configuration management- During the development process, many different versions of each software component are created. If you don't keep track of these versions in a configuration management system, you are liable to include the wrong versions of these components in your system.

Host-target development- Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system). The host and target systems are sometimes of the same type, but often they are completely different.

Course Contents

Reuse:

From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language.

- The only significant reuse or software was the reuse of functions and objects in programming language libraries.
- Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
- An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Course Contents

Reuse Levels:

The abstraction level

At this level, you don't reuse software directly but use knowledge of successful abstractions in the design of your software.

The object level

At this level, you directly reuse objects from a library rather than writing the code yourself.

The component level

Components are collections of objects and object classes that you reuse in application systems.

The system level

At this level, you reuse entire application systems.

Course Contents

Reuse Costs:

1. The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs.
2. Where applicable, the costs of buying the reusable software. For large off-the-shelf systems, these costs can be very high.
3. The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing.
4. The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

Course Contents

Configuration Management:

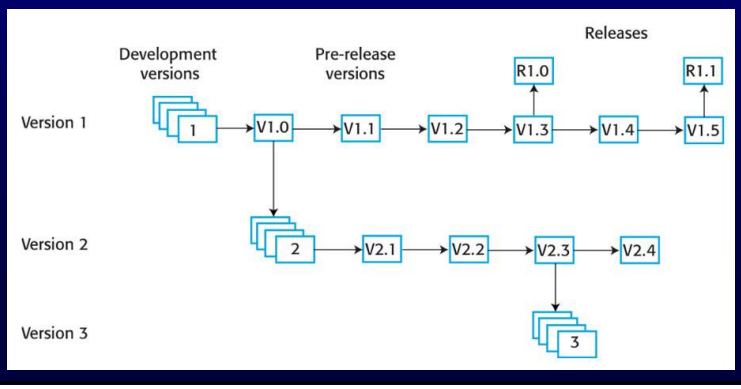
- Configuration management is the name given to the general process of managing a changing software system.
- The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.

Course Contents

Configuration Management:

For large systems, there is never just one 'working' version of a system. There are always several versions of the system at different stages of development. There may be several teams involved in the development of different system versions. GitHub is a highly used software that is typically used for version control. It is helpful when more than just one person is working on a project.

Git is a widely-used example of a distributed VC system.



Course Contents

Configuration Management Activities:

- Version management, where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
- System integration, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- Problem tracking, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

Course Contents

Host-Target Development:

Most software is developed on one computer (the host), but runs on a separate machine (the target).

More generally, we can talk about a development platform and an execution platform.

- A platform is more than just hardware.
- It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.

Development platform usually has different installed software than execution platform; these platforms may have different architectures.

Course Contents

Open-Source Development:

- Open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process
- Its roots are in the Free Software Foundation (www.fsf.org), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish.
- Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code.
- The best-known open source product is, of course, the Linux operating system which is widely used as a server system and, increasingly, as a desktop environment.
- Other important open source products are Java, the Apache web server and the mySQL database management system.

Course Contents

Open source issues:

- Should the product that is being developed make use of open source components?
- Should an open source approach be used for the software's development?
- More and more product companies are using an open source approach to development.
- Their business model is not reliant on selling a software product but on selling support for that product.
- They believe that involving the open source community will allow software to be developed more cheaply, more quickly and will create a community of users for the software

Course Contents

Open source licensing:

- A fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code.
  - Legally, the developer of the code (either a company or an individual) still owns the code. They can place restrictions on how it is used by including legally binding conditions in an open source software license.
  - Some open source developers believe that if an open source component is used to develop a new system, then that system should also be open source.
  - Others are willing to allow their code to be used without this restriction. The developed systems may be proprietary and sold as closed source systems.

Course Contents

License models:

- The GNU General Public License (GPL). This is a so-called 'reciprocal' license that means that if you use open source software that is licensed under the GPL license, then you must make that software open source.
- The GNU Lesser General Public License (LGPL) is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components.
- The Berkley Standard Distribution (BSD) License. This is a non-reciprocal license, which means you are not obliged to re-publish any changes or modifications made to open source code. You can include the code in proprietary systems that are sold.

Course Contents

License management:

- Establish a system for maintaining information about open-source components that are downloaded and used.
- Be aware of the different types of licenses and understand how a component is licensed before it is used.
- Be aware of evolution pathways for components.
- Educate people about open source.
- Have auditing systems in place.
- Participate in the open source community.